

Credit Card Fraud Detection

Parts 6 and 7 Report

Contents

1	Part 6: Evaluation	2
1.1	Loading Tuned Models	2
1.2	Evaluation Metrics Explained	2
1.3	KNN Evaluation	2
1.4	SVM Evaluation	3
1.5	Random Forest Evaluation	4
1.6	Model Comparison Summary	4
1.7	Confusion Matrix Visualization	4
1.8	Overfitting / Underfitting Analysis	5
2	Part 7: Improvement and Experimentation	7
2.1	Experiment 1: Before Scaling vs After Scaling	7
2.2	Experiment 2: Before Tuning vs After Tuning	7
2.3	Experiment 3: Applying SMOTE (Oversampling)	8
2.4	Experiment Comparison Visualization	9
2.5	Final Project Summary	10
2.6	Key Lessons Learned	10

1 Part 6: Evaluation

This section provides a comprehensive evaluation of all three tuned models from Part 5 using multiple metrics, confusion matrices, classification reports, and an overfitting/underfitting analysis.

1.1 Loading Tuned Models

```
1 with open(os.path.join(SCRIPT_DIR, "part5_results.pkl"), "rb") as f:
2     results = pickle.load(f)
3
4 models = results["models"]
5 predictions = results["predictions"]
6 X_test = results["test_data"]["X_test"]
7 y_test = results["test_data"]["y_test"]
```

The tuned models and their predictions saved from Part 5 are loaded. The test set contains 10,000 samples with 39 fraud cases.

1.2 Evaluation Metrics Explained

Before presenting the results, here is what each metric measures:

Metric	Formula	What It Measures
Accuracy	$(TP+TN) / \text{Total}$	Overall correctness. Misleading with imbalanced data.
Precision	$TP / (TP+FP)$	Of all fraud predictions, how many were actually fraud?
Recall	$TP / (TP+FN)$	Of all actual fraud, how many did the model catch?
F1-Score	$2*(P*R)/(P+R)$	Balance between Precision and Recall. Best single metric for imbalanced data.

1.3 KNN Evaluation

```
1 knn_pred = predictions["knn"]
2 acc = accuracy_score(y_test, knn_pred)
3 prec = precision_score(y_test, knn_pred)
4 rec = recall_score(y_test, knn_pred)
5 f1 = f1_score(y_test, knn_pred)
6 cm = confusion_matrix(y_test, knn_pred)
```

Output:

Accuracy: 0.9959
Precision: 0.0000
Recall: 0.0000
F1-Score: 0.0000

Confusion Matrix:

TN=9959 FP=2 FN=39 TP=0

Classification Report:

	precision	recall	f1-score	support
Not Fraud	1.00	1.00	1.00	9961
Fraud	0.00	0.00	0.00	39
accuracy			1.00	10000

Analysis: KNN completely failed to detect any fraud. Even after tuning K=3, the model predicts almost everything as Not Fraud. The 99.59% accuracy is misleading because 99.6% of the data is Not Fraud anyway.

1.4 SVM Evaluation

```
1 svm_pred = predictions["svm"]  
2 # Same metrics calculation as above
```

Output:

Accuracy: 0.9937
Precision: 0.1842
Recall: 0.1795
F1-Score: 0.1818

Confusion Matrix:

TN=9930 FP=31 FN=32 TP=7

Classification Report:

	precision	recall	f1-score	support
Not Fraud	1.00	1.00	1.00	9961
Fraud	0.18	0.18	0.18	39
accuracy			0.99	10000

Analysis: SVM detected 7 out of 39 fraud cases (17.9% recall). Thanks to `class_weight="balanced"` from tuning, the model now pays attention to fraud. However, it also produced 31 false alarms (FP=31), meaning it flagged 31 legitimate transactions as fraud.

1.5 Random Forest Evaluation

Output:

Accuracy: 0.9874

Precision: 0.1282

Recall: 0.3846

F1-Score: 0.1923

Confusion Matrix:

TN=9859 FP=102 FN=24 TP=15

Classification Report:

	precision	recall	f1-score	support
Not Fraud	1.00	0.99	0.99	9961
Fraud	0.13	0.38	0.19	39
accuracy			0.99	10000

Analysis: Random Forest detected 15 out of 39 fraud cases (38.5% recall). This is the best recall among all models. However, precision is low (12.8%) because it generated 102 false alarms. This means for every real fraud detected, about 7 legitimate transactions were wrongly flagged.

1.6 Model Comparison Summary

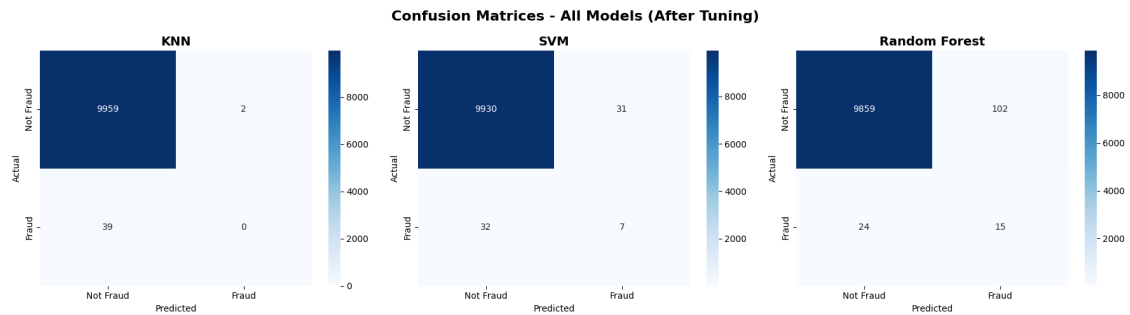
Model	Accuracy	Precision	Recall	F1	TP/39
KNN	0.9959	0.0000	0.0000	0.0000	0
SVM	0.9937	0.1842	0.1795	0.1818	7
Random Forest	0.9874	0.1282	0.3846	0.1923	15

Best model by F1-Score: **Random Forest (F1 = 0.1923)**.

1.7 Confusion Matrix Visualization

```
1 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
2 for i, (name, key) in enumerate(zip(model_names, model_keys)):
3     cm = confusion_matrix(y_test, predictions[key])
4     sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
5                 ax=axes[i],
6                 xticklabels=["Not Fraud", "Fraud"],
7                 yticklabels=["Not Fraud", "Fraud"])
8     axes[i].set_title(name)
9 plt.savefig("plots/confusion_matrices.png")
```

This generates a side-by-side heatmap of all three confusion matrices, making it easy to visually compare model performance.



1.8 Overfitting / Underfitting Analysis

```

1 # For each model, compare train performance vs test performance
2 y_train_pred = model.predict(X_train)
3 train_f1 = f1_score(y_train, y_train_pred)
4 test_f1 = f1_score(y_test, y_test_pred)
5 diff = train_f1 - test_f1

```

Definitions:

Condition	Meaning
Overfitting	Model performs much better on training data than test data. It has memorized the training data instead of learning general patterns.
Underfitting	Model performs poorly on both training and test data. It has not learned enough from the data.
Good Fit	Model performs similarly well on both training and test data.

Output:

Model	Train F1	Test F1	Diff	Diagnosis
KNN	0.3364	0.0000	0.3364	Overfitting
SVM	0.9086	0.1818	0.7268	Overfitting
Random Forest	0.5379	0.1923	0.3456	Overfitting

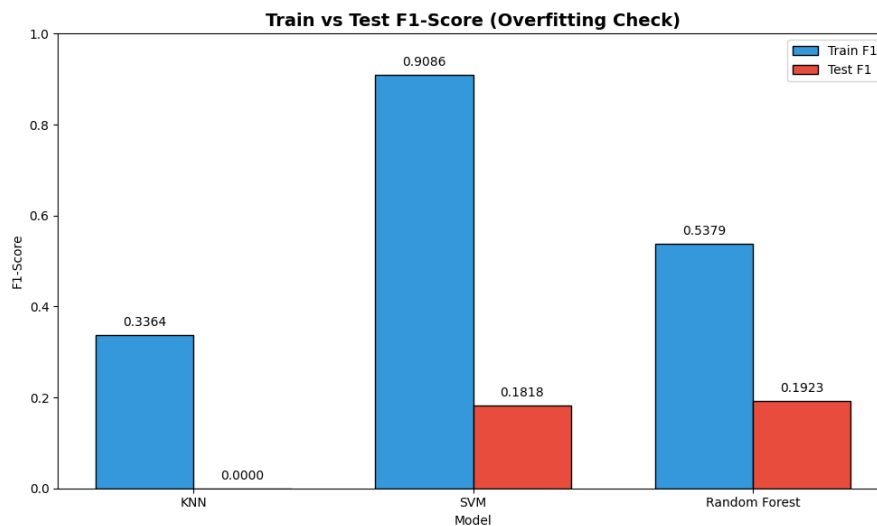
Analysis:

- **KNN:** Train F1 is 0.34 but Test F1 is 0.00. The model learned some patterns from training data but cannot apply them to new data.

- **SVM:** Most extreme overfitting. Train F1 is 0.91 but Test F1 is only 0.18. The model memorized training patterns very well but fails on unseen data.
- **Random Forest:** Train F1 is 0.54, Test F1 is 0.19. Still overfitting but less extreme than SVM.

Why all models overfit:

1. The extreme class imbalance (0.58% fraud) makes it very hard to learn reliable fraud patterns.
2. The training sample is only 30,000 rows with approximately 174 fraud cases, which is very few examples to learn from.
3. Fraud patterns may be complex and require more data or advanced techniques to generalize.



2 Part 7: Improvement and Experimentation

This section conducts three experiments to improve model performance and compares the results before and after each improvement.

2.1 Experiment 1: Before Scaling vs After Scaling

```
1 # Train Random Forest on UNSCALED data
2 rf_unscaled = RandomForestClassifier(
3     n_estimators=200, max_depth=10,
4     class_weight="balanced", random_state=42
5 )
6 rf_unscaled.fit(X_train_unsub, y_train_unsub)
7 pred_unscaled = rf_unscaled.predict(X_test_unsub)
8
9 # Compare with SCALED version from Part 5
10 pred_scaled = part5["predictions"]["rf"]
```

What is scaling?

Scaling transforms features to a common range. Without scaling, features like **amt** (range: 1–29000) dominate over **gender** (range: 0–1) in distance-based calculations.

Output:

	Before Scaling	After Scaling

Accuracy	0.9869	0.9874
Precision	0.1230	0.1282
Recall	0.3846	0.3846
F1-Score	0.1863	0.1923
Fraud Detected (TP)	15	15

Analysis: For Random Forest, scaling has minimal impact because tree-based models are not sensitive to feature scales. Both versions detected 15 fraud cases. However, scaling is critical for KNN and SVM which rely on distance calculations.

2.2 Experiment 2: Before Tuning vs After Tuning

This compares Part 4 (default parameters) with Part 5 (tuned parameters + `class_weight`).

```
1 # Part 4: Default models
2 knn_before = part4["predictions"]["knn"] # KNN(n_neighbors=5)
3 svm_before = part4["predictions"]["svm"] # SVC(kernel="rbf")
4 rf_before = part4["predictions"]["rf"]   # RF(n_estimators=100)
```

```

5
6 # Part 5: Tuned models
7 knn_after = part5["predictions"]["knn"] # KNN(n_neighbors=3)
8 svm_after = part5["predictions"]["svm"] # SVC(C=10, balanced)
9 rf_after = part5["predictions"]["rf"]   # RF(200, depth=10,
    balanced)

```

Output:

Model	Before F1	After F1	Before TP	After TP
KNN	0.0000	0.0000	0	0
SVM	0.0000	0.1818	0	7
Random Forest	0.2273	0.1923	5	15

Analysis:

- **KNN:** No improvement. KNN lacks a class weighting mechanism, so tuning K alone cannot fix the imbalance problem.
- **SVM:** Major improvement. Adding `class_weight="balanced"` and tuning `C=10` with RBF kernel allowed SVM to detect 7 fraud cases instead of 0.
- **Random Forest:** The F1-Score decreased slightly (0.23 to 0.19) but TP tripled from 5 to 15. This is because `class_weight="balanced"` increased recall at the cost of precision (more false alarms).

2.3 Experiment 3: Applying SMOTE (Oversampling)

```

1 from imblearn.over_sampling import SMOTE
2
3 smote = SMOTE(random_state=42)
4 X_train_smote, y_train_smote = smote.fit_resample(
5     X_train, y_train
6 )

```

What is SMOTE?

SMOTE (Synthetic Minority Over-sampling Technique) creates synthetic fraud samples by interpolating between existing fraud cases. This balances the dataset so models can learn fraud patterns better.

Before SMOTE: 29,826 Not Fraud, 174 Fraud

After SMOTE: 29,826 Not Fraud, 29,826 Fraud

SMOTE generated 29,652 synthetic fraud samples to match the number of non-fraud cases.

```
1 # Train all 3 models on SMOTE-balanced data
2 knn_smote = KNeighborsClassifier(n_neighbors=3)
3 knn_smote.fit(X_train_smote, y_train_smote)
4
5 svm_smote = SVC(kernel="rbf", C=10, random_state=42)
6 svm_smote.fit(X_train_smote, y_train_smote)
7
8 rf_smote = RandomForestClassifier(
9     n_estimators=200, max_depth=10, random_state=42
10 )
11 rf_smote.fit(X_train_smote, y_train_smote)
```

Output:

Model	Without SMOTE F1	With SMOTE F1	SMOTE TP

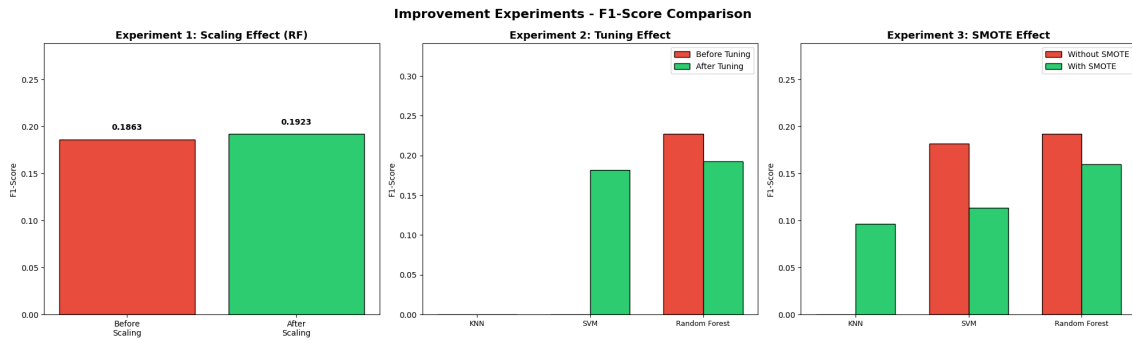
KNN	0.0000	0.0963	9
SVM	0.1818	0.1136	5
Random Forest	0.1923	0.1600	20

Analysis:

- **KNN with SMOTE:** Detected 9 fraud cases (previously 0). SMOTE solved KNN's main problem by providing balanced training data, since KNN has no built-in class weighting.
- **SVM with SMOTE:** Detected 5 fraud (previously 7 with `class_weight`). SMOTE was less effective than `class_weight` for SVM in this case.
- **RF with SMOTE:** Detected 20 fraud cases, the highest count across all experiments. However, F1-Score is lower than tuned RF because of increased false alarms.

2.4 Experiment Comparison Visualization

```
1 fig, axes = plt.subplots(1, 3, figsize=(20, 6))
2 # Plot 1: Scaling effect
3 # Plot 2: Tuning effect (before vs after)
4 # Plot 3: SMOTE effect (without vs with)
5 plt.savefig("plots/improvement_experiments.png")
```



2.5 Final Project Summary

Improvement Journey:

Stage	Action	Result
Part 4	Base models (default settings)	KNN: TP=0, SVM: TP=0, RF: TP=5
Part 5	Hyperparameter tuning + class_weight	KNN: TP=0, SVM: TP=7, RF: TP=15
Part 7	SMOTE oversampling	KNN: TP=9, SVM: TP=5, RF: TP=20

Best Overall Model: Random Forest (Tuned)

Accuracy: 0.9874

Precision: 0.1282

Recall: 0.3846

F1-Score: 0.1923

Fraud Detected: 15 out of 39

2.6 Key Lessons Learned

1. **Accuracy is misleading** with imbalanced data. A model predicting everything as Not Fraud achieves 99.4% accuracy but detects zero fraud.
2. **class_weight="balanced"** is the most effective single technique for handling class imbalance, especially for SVM and Random Forest.
3. **SMOTE oversampling** helps models without built-in class weighting (like KNN) by generating synthetic minority samples.
4. **F1-Score** is the correct metric for evaluating fraud detection models, as it balances precision (avoiding false alarms) and recall (catching fraud).

5. **Scaling** is important for distance-based models (KNN, SVM) but has minimal effect on tree-based models (Random Forest).
6. **Random Forest** generally handles imbalanced data best among the three models tested.
7. **All models showed overfitting**, which is expected with only 174 fraud training samples. More data or advanced techniques could help.